

DevOps. Automatización y monitorización de infraestructuras

Unidad 04. Automatización remota con Salt y monitorización de sistemas - Laboratorio 01



Autor: Sergi García Barea

Actualizado Mayo 2026



Licencia



Reconocimiento – NoComercial – CompartirIgual (BY-NC-SA):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Unidad 04. Automatización remota con Salt y monitorización de sistemas - Laboratorio 01	2
1. Explicación ¿Qué haremos en este caso práctico?	2
2. Objetivos	2
3. El modelo mental	2
4. Topología del proyecto	3
5. Fase de ejecución: despliegue paso a paso	3
Paso 1: Orquestación de red (El cimiento)	4
Paso 2: Infraestructura via Terraform (La Estructura)	4
Paso 3: Provisión del Salt Master (El Capataz)	5
Paso 4: Provisión de los Salt Minions (Los operarios)	6
Paso 5: Instalación del cerebro de observabilidad (Prometheus)	7
Paso 6: Instalación del Visualizador (Grafana)	8
Paso 7: Configuración automatizada (El Dashboard Final)	8
6. [OPCIONAL] Integración avanzada: Puente con UD03	9
7. Pruebas manuales	10
La prueba visual (Centro de mando)	11
8. Operaciones de limpieza	12
9. Problemas comunes	12

UNIDAD 04. AUTOMATIZACIÓN REMOTA CON SALT Y MONITORIZACIÓN DE SISTEMAS - LABORATORIO 01

1. EXPLICACIÓN ¿QUÉ HAREMOS EN ESTE CASO PRÁCTICO?

En este laboratorio vamos a construir un ecosistema de automatización y monitorización completo para gobernar nuestra infraestructura.

Dejaremos atrás la configuración manual y desplegaremos un clúster de contenedores utilizando Infraestructura como Código (Terraform). A continuación, instalaremos y configuraremos un sistema centralizado de ejecución remota (SaltStack) y montaremos un "cerebro" de observabilidad basado en Prometheus y Grafana para visualizar en tiempo real el consumo de recursos y la salud de la red. Finalmente, **DE FORMA TOTALMENTE OPCIONAL**, conectaremos las máquinas virtuales de la Unidad 03 a este nuevo panel de control.

2. OBJETIVOS

Al terminar este caso práctico, habrás logrado:

- **Orquestación:** Levantar una infraestructura multicontenedor de forma declarativa con Terraform, incluyendo redes aisladas y volúmenes persistentes.
- **Gestión centralizada:** Desplegar una arquitectura Master-Minion con SaltStack para controlar múltiples servidores simultáneamente.
- **Observabilidad:** Desplegar Prometheus para recolectar métricas mediante el modelo *Pull*, apoyándose en Node Exporter, cAdvisor y Blackbox Exporter.
- **Visualización dinámica:** Configurar cuadros de mando visuales y automatizados en Grafana mediante llamadas a su API REST.
- **Integración híbrida:** Unificar contenedores Docker y máquinas virtuales Vagrant bajo un mismo paraguas de monitorización.

3. EL MODELO MENTAL

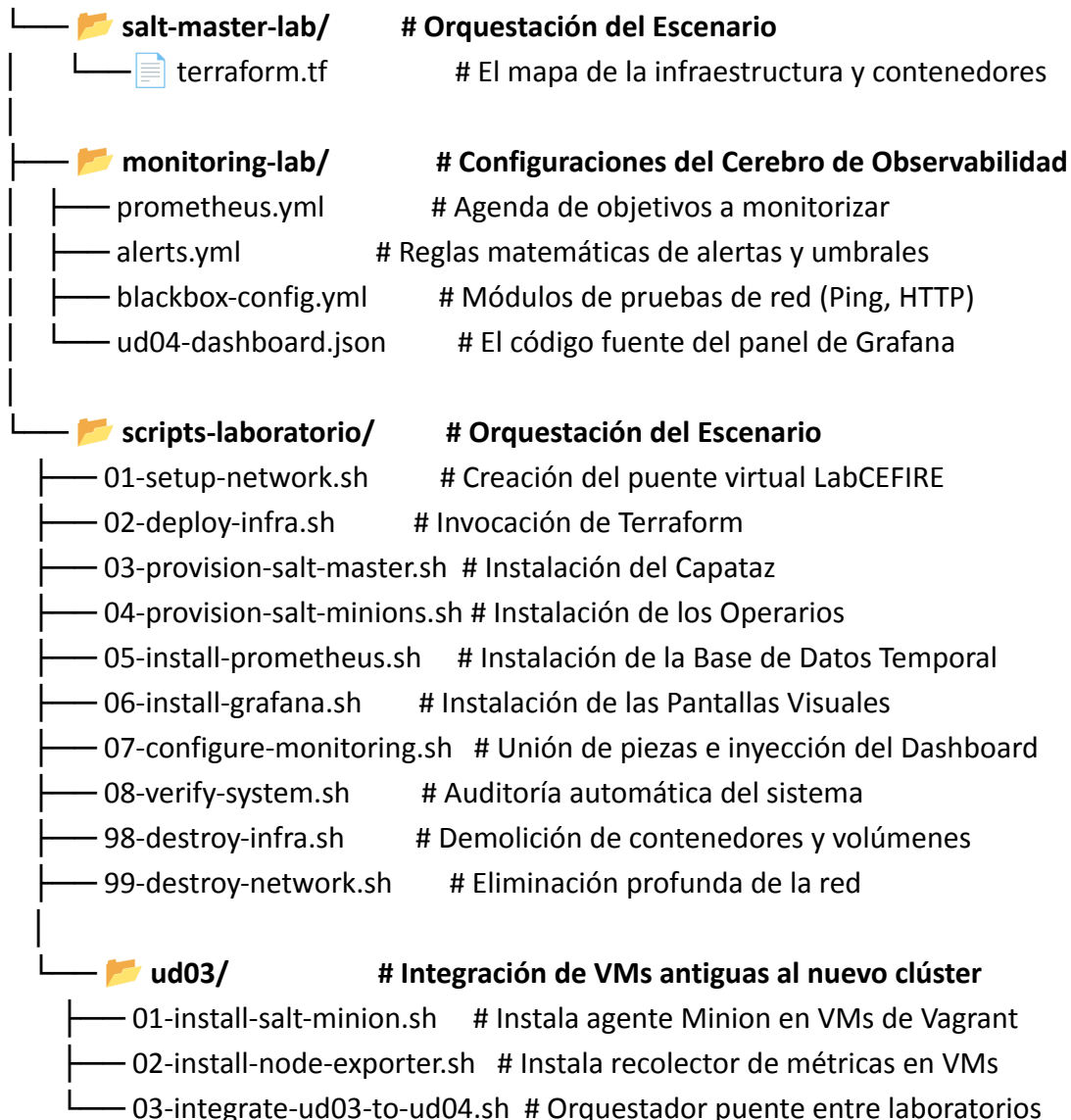
Para entender el ecosistema que vamos a construir, imagina que estás diseñando el centro de mando de una fábrica inteligente:

1. **Los Cimientos y el Edificio (Terraform):** Define cuántas naves industriales necesitas, sus almacenes (volúmenes) y los pasillos de comunicación (red bridge LabCEFIRE).
2. **El Capataz y los Operarios (SaltStack):** El *Master* es el capataz que da las órdenes por megafonía. Los *Minions* son los operarios en cada nave que tienen un walkie-talkie cifrado esperando instrucciones.
3. **El Centro de Seguridad (Prometheus y Exporters):** Los *Exporters* son las cámaras y sensores de temperatura instalados en las naves. *Prometheus* es el servidor de grabación que recoge los videos y datos cada 15 segundos.
4. **Las Pantallas de Control (Grafana):** Es el videowall gigante donde el director de la fábrica ve gráficos en tiempo real con la salud de toda la instalación.

4. TOPOLOGÍA DEL PROYECTO

La infraestructura se basa en esta estructura de archivos para garantizar un orden estricto de ejecución:

Unidad-04-Salt-Observabilidad/Laboratorio/



5. FASE DE EJECUCIÓN: DESPLIEGUE PASO A PASO

Antes de poner las manos en el teclado, asegúrate de cumplir con los requisitos previos: tener instalado Docker y Terraform en tu sistema anfitrión.

Para evitar errores de dependencias, sigue este orden estrictamente. Opcionalmente, puedes ejecutar `bash run-all.sh` para que un orquestador maestro haga todo el proceso por ti de forma automatizada, pero como buenos arquitectos, vamos a diseccionar qué ocurre en cada fase.

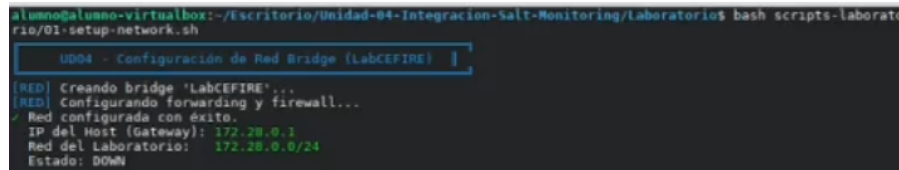
Abre tu terminal y sitúate en la carpeta **scripts-laboratorio/**.

Paso 1: Orquestación de red (El cimiento)

Antes de levantar ningún servidor o contenedor, necesitamos construir la "carretera" por la que se comunicarán. En lugar de depender de las redes dinámicas y automáticas que crea Docker por defecto, vamos a construir un segmento de red estático y completamente aislado.

```
bash scripts-laboratorio/01-setup-network.sh
```

- ¿Qué ocurre bajo el capó?



```
alumno@alumno-virtualbox:~/Escritorio/Unidad-04-Integracion-Salt-Monitoring/Laboratorio$ bash scripts-laboratorio/01-setup-network.sh
UD04 - Configuración de Red Bridge (LabCE FIRE)

[RED] Creando bridge 'LabCE FIRE'...
[RED] Configurando forwarding y firewall...
✓ Red configurada con éxito.
IP del Host (Gateway): 172.28.0.1
Red del Laboratorio: 172.28.0.0/24
Estado: DOWN
```

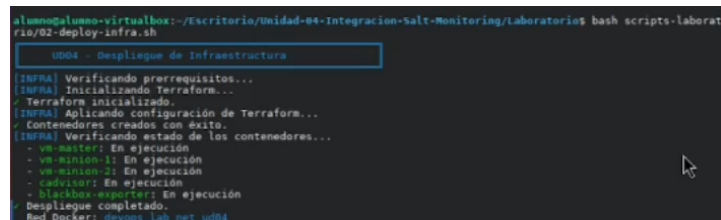
- Este script no interactúa con Docker ni con Terraform todavía. Habla directamente con el subsistema de red del Kernel de Linux de tu equipo anfitrión (Host) para alterar su topología.
- Comandos y procesos clave:
 - **ip link add name LabCE FIRE type bridge:** Le ordenamos al Kernel que instancie un conmutador (switch) lógico de Capa 2. Al darle un nombre explícito (LabCE FIRE), nos aseguramos de poder "enchufar" más adelante tanto contenedores Docker como máquinas virtuales Vagrant (de la UD03) al mismo dominio de colisión.
 - **ip addr add 172.28.0.1/24 dev LabCE FIRE:** Le asignamos una dirección IP estática a la interfaz virtual de tu propio ordenador físico. Esta IP (.1) es vital, ya que actuará como la puerta de enlace (Gateway) predeterminada para todos los servidores del laboratorio.

Paso 2: Infraestructura via Terraform (La Estructura)

Una vez preparado el terreno de red, desplegamos el esqueleto computacional: los contenedores y sus volúmenes de almacenamiento persistente asociados.

```
bash scripts-laboratorio/02-deploy-infra.sh
```

- ¿Qué ocurre bajo el capó?



```
alumno@alumno-virtualbox:~/Escritorio/Unidad-04-Integracion-Salt-Monitoring/Laboratorio$ bash scripts-laboratorio/02-deploy-infra.sh
UD04 - Despliegue de Infraestructura

[INFRA] Verificando prerequisites...
[INFRA] Inicializando Terraform...
Terraform inicializado.
[INFRA] Aplicando configuración de Terraform...
Contenedores creados con éxito.
[INFRA] Verificando estado de los contenedores...
- vm-master: En ejecución
- vm-minion-1: En ejecución
- vm-minion-2: En ejecución
- cadvisor: En ejecución
- blackbox-exporter: En ejecución
Despliegue completado.
Red Docker: docker_lab_net_ud04
```

- El script actúa como un envoltorio (wrapper) de seguridad. Primero, verifica y purga cualquier contenedor o estado residual de ejecuciones anteriores para garantizar un despliegue limpio (idempotencia). Acto seguido, invoca los comandos maestros de

Infraestructura como Código: terraform init (para inicializar y descargar el proveedor de Docker) y terraform apply -auto-approve (para traducir nuestro código declarativo en llamadas directas a la API del demonio de Docker).

🔍 2.1. El Plano Maestro: Diseccionando terraform.tf

Al igual que el Vagrantfile gobernaba las máquinas virtuales en la unidad anterior, el archivo **terraform.tf** es nuestra Única Fuente de Verdad para este ecosistema de contenedores. Estas son sus directivas críticas y su razón de ser:

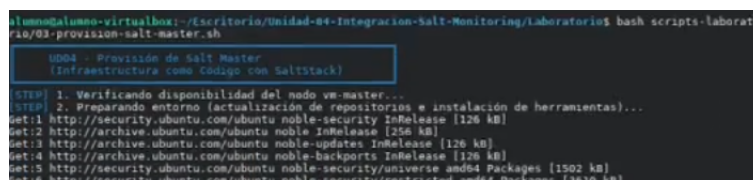
- **El enlace de red (resource "docker_network" "internal_net")**: Esta directiva no se limita a crear una red aislada genérica en Docker. Mediante sus opciones avanzadas, acopla lógicamente la red de los contenedores directamente al puente físico LabCEFIRE que creamos en el Paso 1. Esto permite que los contenedores sean "ciudadanos de pleno derecho" en el segmento 172.28.0.0/24.
- **Persistencia de Datos (resource "docker_volume")**: Por diseño, los contenedores son efímeros; si se destruyen o reinician, la información de su capa de escritura desaparece. Terraform declara volúmenes persistentes físicos en el host y los ancla a rutas críticas de Prometheus y SaltStack. De esta forma, si el contenedor maestro muere y es recreado, el historial de métricas almacenadas y las claves criptográficas (PKI) previamente aceptadas sobrevivirán al proceso.
- **La inyección al vuelo (upload)**: En lugar de depender de complejas monturas de carpetas compartidas (bind mounts) para inyectar configuraciones, Terraform es capaz de introducir físicamente los archivos prometheus.yml y alerts.yml dentro del sistema de archivos del contenedor vm-master en el mismo milisegundo en que lo instancia, dejándolo listo para ejecutar.
- **Direccionamiento estático (ipv4_address)**: En un entorno estricto de observabilidad y automatización, depender de un servidor DHCP para asignar IPs dinámicas es un riesgo inaceptable. Terraform fija IPs estáticas e inamovibles para cada servicio: el vm-master en la 172.28.0.40, cadvisor en la .41, blackbox-exporter en la .42, y los operarios (vm-minion-1 y 2) en las .43 y .44. Esto garantiza de forma absoluta que los Minions siempre sabrán dónde encontrar a su jefe y que Prometheus sabrá exactamente a qué puertas llamar para recolectar las métricas.

Paso 3: Provisión del Salt Master (El Capataz)

Instalamos el cerebro de la automatización centralizada.

```
bash scripts-laboratorio/03-provision-salt-master.sh
```

- ¿Qué ocurre bajo el capó?



```
alumno@alumno-virtualbox: ~/Escritorio/Unidad-04-Integracion-Salt-Monitoring/laboratorio$ bash scripts-laboratorio/03-provision-salt-master.sh
0304 - Provisión de Salt Master
(Infraestructura como Código con SaltStack)

STEP 1: Verificando disponibilidad del nodo vm-master...
STEP 2: Preparando entorno (actualización de repositorios e instalación de herramientas)...
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1502 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [3516 kB]
```

- Dado que estamos trabajando con contenedores Docker que no exponen el puerto 22 por defecto, no utilizamos SSH para la configuración inicial. En su lugar, el script utiliza el

socket del demonio de Docker (docker exec vm-master bash -c "...") para inyectar un flujo de comandos directamente en la terminal del contenedor de forma completamente desatendida (export DEBIAN_FRONTEND=noninteractive).

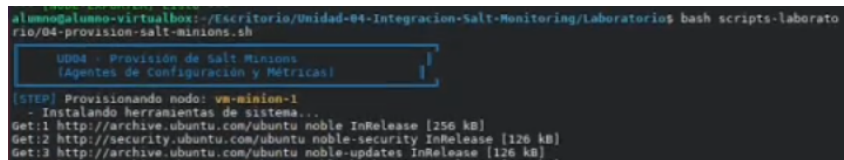
- **La explicación del arquitecto:**
 - **Autenticidad del software:** Primero, el script descarga e instala la clave GPG oficial de Broadcom/SaltProject para asegurar que los paquetes no han sido manipulados maliciosamente. Acto seguido, instala el paquete base salt-master.
 - **Seguridad por defecto (Zero Trust):** Se inyecta un bloque crítico de configuración en el archivo maestro (/etc/salt/master): la directiva auto_accept: false. En un entorno de producción, si esta directiva estuviera activada, cualquier máquina no autorizada en la red que se hiciese pasar por un Minion tendría acceso al bus de control. Al forzarla a false, garantizamos que ningún nodo se pueda unir a nuestra red de automatización sin la autorización criptográfica explícita de un administrador humano.
 - **Arranque sin Systemd:** Al ser un contenedor Docker puro (basado en Ubuntu), no disponemos del gestor de servicios moderno systemd (ni del comando systemctl). Para solucionar esto, el script arranca el demonio de Salt manualmente en segundo plano utilizando nohup /usr/bin/salt-master -d, redirigiendo la salida a un archivo log para que el proceso sobreviva una vez que el script termine su ejecución.

Paso 4: Provisión de los Salt Minions (Los operarios)

Una vez que el capataz está listo y escuchando en la red, desplegamos los agentes de automatización y los sensores de métricas en los nodos de trabajo.

```
bash scripts-laboratorio/04-provision-salt-minions.sh
```

- **¿Qué ocurre bajo el capó?**



- El script realiza un bucle de control (for m in \$MINIONS) para iterar sobre los contenedores **vm-minion-1** y **vm-minion-2**, aprovisionándolos de forma secuencial.
- **Comandos y procesos clave:**
 - **Instalación dual (Control y Observabilidad):** Se inyectan dos piezas de software fundamentales en paralelo. Por un lado, el agente **salt-minion** para obedecer órdenes, y por otro, el demonio **prometheus-node-exporter** para abrir las entrañas del sistema (CPU, RAM, uso de disco) y exponerlas por HTTP en el puerto 9100.
 - **El registro (echo "master: 172.28.0.40" > /etc/salt/minion):** A diferencia de otros sistemas de orquestación donde el servidor escanea activamente la red buscando clientes, Salt tiene un enfoque inverso. Le indicamos explícitamente a los Minions la IP estática inamovible de su jefe. Al arrancar, el Minion buscará

proactivamente al Master en la red.

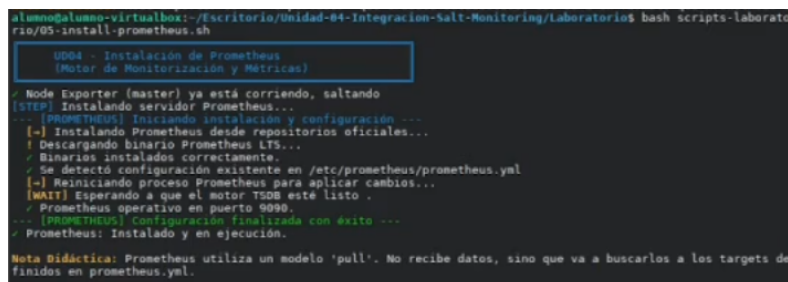
- **El apretón de manos criptográfico (PKI):** Tras encender los Minions, estos generan sus claves de seguridad RSA y envían la clave pública al Master solicitando acceso. El script hace una pausa estratégica de 5 segundos para dar tiempo a que los paquetes crucen la red virtual y, a continuación, ejecuta automáticamente **`docker exec vm-master salt-key -A -y`**. Este comando acepta formalmente en el servidor todas las claves que estaban pendientes (Unaccepted), estableciendo el canal de comunicación cifrado mediante AES.
- **Auditoría de conectividad:** Inmediatamente después del registro, para confirmar que el bus de mensajes de alta velocidad (ZeroMQ) funciona a la perfección, el script lanza el comando global **`salt '*' test.ping`**. Esto obliga a todos los nodos registrados en la infraestructura a responder afirmativamente en milisegundos, confirmando que el entorno está vivo y listo para operar.

Paso 5: Instalación del cerebro de observabilidad (Prometheus)

Instalamos el motor de almacenamiento de métricas, conocido técnicamente como una Time-Series Database (Base de datos de series temporales).

```
bash scripts-laboratorio/05-install-prometheus.sh
```

- **¿Qué ocurre bajo el capó?**



```
alumn@galena-virtualbox: ~/Escritorio/Unidad-04-Integracion-Salt-Monitoring/Laboratorio5 bash scripts-laboratorio/05-install-prometheus.sh
UD04 - Instalación de Prometheus
(Motor de Monitorización y Métricas)

Node Exporter (master) ya está corriendo, saltando
[STEP] Instalando servidor Prometheus...
[PROMETHEUS] Iniciando instalación y configuración ---
[-] Instalando Prometheus desde repositorios oficiales...
! Descargando binario Prometheus LTS...
✓ Binarios instalados correctamente.
✓ Se detectó configuración existente en /etc/prometheus/prometheus.yml
[-] Reiniciando proceso Prometheus para aplicar cambios...
[WAIT] Esperando a que el motor TSDB esté listo .
✓ Prometheus operativo en puerto 9090.
--- [PROMETHEUS] Configuración finalizada con éxito ---
✓ Prometheus: Instalado y en ejecución.

Nota Didáctica: Prometheus utiliza un modelo 'pull'. No recibe datos, sino que va a buscarlos a los targets de
finidas en prometheus.yml.
```

- Al estar dentro de un contenedor en crudo, el script descarga el binario, prepara su estructura de carpetas (**`/etc/prometheus`** y **`/var/lib/prometheus`**) y lo lanza en segundo plano atacando el archivo maestro **`prometheus.yml`** que Terraform inyectó en el Paso 2.
- **La explicación del arquitecto:** A diferencia de las aplicaciones web clásicas, **Prometheus** no requiere instalar un motor pesado de bases de datos relacionales (como MySQL o PostgreSQL) de fondo. Es un binario ultraligero, escrito en Go y completamente autocontenido, diseñado para un único propósito: procesar datos a velocidades extremas.
- **El modelo Pull en acción:** Desde el milisegundo en que el proceso arranca, **Prometheus** no se queda pasivo esperando a que le envíen datos. Se convierte en un rastreador activo; comienza a "raspar" (hacer pull o peticiones GET) de forma agresiva a los puertos de nuestros nodos cada 15 segundos exactos. **Llama a la puerta del puerto 9100 para hablar con los Node Exporters, al 8080 para interrogar a cAdvisor sobre Docker, y al 9115 para pedirle métricas a Blackbox Exporter**, guardando cada dato crudo en su disco persistente.

Paso 6: Instalación del Visualizador (Grafana)

Añadimos la capa gráfica interactiva al ecosistema para dar sentido humano a los millones de datos numéricos recolectados.

```
bash scripts-laboratorio/06-install-grafana.sh
```

- ¿Qué ocurre bajo el capó?

```
alumno@alumno-virtualbox: ~/Escritorio/Unidad-04-Integracion-Salt-Monitoring/Laboratorio$ bash scripts-laboratorio/06-install-grafana.sh
UD04 - Instalación de Grafana
(Plataforma de Visualización y Observabilidad)

[STEP] Instalando servidor Grafana (via OSS)...
--- [GRAFANA] Iniciando instalación y configuración ---
[-] Configurando repositorio oficial de Grafana OSS...
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 https://apt.grafana.com stable InRelease [7680 B]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2412 kB]
Get:7 https://apt.grafana.com stable/main amd64 Packages [547 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2145 kB]
Hit:9 https://packages.broadcom.com/artifactory/saltproject-deb stable InRelease
Fetched 5491 kB in 1s (3984 kB/s)
Reading package lists...
[-] Instalando paquete 'grafana'...
```

- El script instala el paquete oficial de Grafana y sobrescribe inmediatamente su archivo central (*/etc/grafana/grafana.ini*). Esta inyección de configuración es vital para forzar las credenciales maestras por defecto (**admin/admin**), fijar la escucha en el puerto TCP 3000 y deshabilitar registros no autorizados, todo de forma desatendida.
- **La trampa (El bucle de espera inteligente):** Un error muy común al automatizar infraestructuras es intentar configurar un servicio que aún no ha terminado de arrancar en memoria. Grafana utiliza una base de datos interna (SQLite) que necesita crear y migrar decenas de tablas la primera vez que se ejecuta. Para evitar condiciones de carrera (race conditions), el script implementa un bucle for/while que sondea repetidamente el endpoint de diagnóstico interno ***http://localhost:3000/api/health***. El orquestador pausa la ejecución y no devuelve el "OK" por pantalla hasta que Grafana certifica oficialmente que ha terminado su configuración inicial y está sano.

Paso 7: Configuración automatizada (El Dashboard Final)

Unimos el motor de bases de datos con los gráficos interactivos sin tocar la interfaz web.

```
bash scripts-laboratorio/07-configure-monitoring.sh
```

- ¿Qué ocurre bajo el capó?

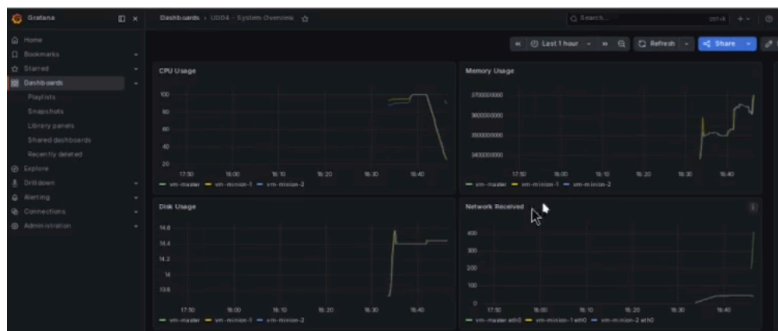
```
alumno@alumno-virtualbox: ~/Escritorio/Unidad-04-Integracion-Salt-Monitoring/Laboratorio$ bash scripts-laboratorio/07-configure-monitoring.sh
UD04 - Configuración de Monitorización

[CONFIG] Actualizando configuración de Prometheus (targets y alertas)...
Successfully copied 2.58kB (transferred 4.6kB) to vm-master:/etc/prometheus/prometheus.yml
Successfully copied 7.08kB (transferred 8.7kB) to vm-master:/etc/prometheus/alerts.yml
Prometheus configuration reloaded via SIGHUP
[CONFIG] Configuración de Prometheus actualizada
[CONFIG] Esperando a que los Node Exporters envíen métricas...
[CONFIG] Esperando a que Prometheus tenga datos de Node Exporters...
[CONFIG] Prometheus tiene 8 targets activos
[CONFIG] Esperando métrica: node_filesystem_size_bytes...
[CONFIG] Métrica node_filesystem_size_bytes disponible
[CONFIG] Verificando acceso a la API de Grafana...
[CONFIG] Autenticación en Grafana lista
[CONFIG] Configurando Grafana datasource...
[CONFIG] Eliminando dashboard anterior si existe...
[CONFIG] Datasource Prometheus configurado
[CONFIG] Esperando que Prometheus recolecte métricas iniciales...
```

- El script realiza una orquestación en dos frentes. Primero, copia la agenda de objetivos definitiva (***prometheus.yml***) dentro del contenedor maestro y envía una señal ***SIGHUP***

(kill -HUP) al proceso interno de Prometheus.

- Este es un truco obliga al motor a recargar su nueva configuración "en caliente", sin detener el demonio ni perder un solo segundo de recolección de datos.
- Una vez que la base de datos está configurada, el script negocia la autenticación con Grafana y ejecuta las inyecciones de red.
- **La explicación del arquitecto:** Esta fase representa la verdadera madurez DevOps. En un entorno IT tradicional, un administrador entra ahora a Grafana con su navegador, haría clic en "Añadir Datasource", configuraría la IP a mano y pasaría horas creando o importando paneles. **Nosotros vamos a prescindir de la interfaz gráfica e interactuaremos directamente con la API REST de Grafana.**
- **Comandos e inyecciones clave:**
 - **Creación del datasource:** El script lanza una petición de red silenciosa (**curl -X POST**) autenticada contra la API, inyectando un bloque (payload) en formato JSON. Este código le indica a Grafana que su fuente de datos por defecto es de tipo Prometheus y reside en **http://localhost:9090**.
 - **Inyección del cuadro de mando (Dashboard):** Acto seguido, ejecuta un segundo **curl -X POST** masivo. En esta petición envía la estructura completa del panel ("UD04 - System Overview").
 - En Grafana, un Dashboard no es más que un inmenso archivo JSON que describe geometrías, colores y consultas PromQL. Al inyectarlo por la API, Grafana dibuja y despliega instantáneamente las gráficas de CPU, Memoria, Red y latencia de los contenedores Docker, dejando el panel de mando 100% operativo sin intervención humana.



6. [OPCIONAL] INTEGRACIÓN AVANZADA: PUENTE CON UD03

Una verdadera infraestructura DevOps no consiste en sistemas aislados ni en silos tecnológicos. Si conservas las máquinas virtuales creadas con Vagrant en el laboratorio de la Unidad 03 (El Servidor FOG y la Golden Image) y están encendidas, puedes anexarlas a este nuevo centro de mando.

Suponiendo que ambas unidades están descargadas en la misma carpeta base, dirígete a la raíz del laboratorio de la UD03 y ejecuta el orquestador puente:

```
cd ../Unidad-03-FOG-PXE/Laboratorio/  
../Unidad-04-Salt-Monitoring/Laboratorio/scripts-laboratorio/ud03/03-integrate-ud03-to-ud04.sh
```

- **¿Qué ocurre bajo el capó?** El orquestador localiza las máquinas virtuales Legacy (gobernadas por Vagrant y VirtualBox) en tu sistema y realiza un asalto controlado a través de la red puente física que comparten. Entra por SSH de forma totalmente automatizada (*vagrant ssh -c*) e inyecta rutinas de configuración desatendidas en el sistema operativo huésped.
- **La explicación del arquitecto:** En un centro de datos real, rara vez administrarás una infraestructura basada exclusivamente en contenedores o puramente en máquinas virtuales; lo habitual es lidiar con arquitecturas híbridas. Este script demuestra cómo unificar recursos heterogéneos. Estamos conectando hipervisores tradicionales (UD03) con orquestadores modernos (Terraform/Docker de la UD04) utilizando SaltStack como bus de gestión y Prometheus como capa de observabilidad global.
- **Comandos y procesos clave:**
 - **Inyección de agentes:** Instala el agente salt-minion para recibir órdenes y el prometheus-node-exporter para abrir las métricas en las máquinas de la UD03 (FOG Server y Golden Image) sin detener ni afectar a los servicios de clonación que ya tenían configurados.
 - **Enrutamiento de control:** Sobrescribe la configuración del agente en las máquinas virtuales para indicarles que su nuevo jefe reside en la IP 172.28.0.40 (la dirección de nuestro contenedor maestro de Docker).
 - **Automatización de confianza (PKI):** Tras la instalación, el script detiene su ejecución unos segundos, esperando a que las máquinas virtuales generen sus claves criptográficas y las envíen a través de la red LabCEFIRE. Acto seguido, envía un comando al contenedor vm-master (*salt-key -A -y*) para que acepte formalmente estos nuevos certificados, validando la comunicación.
 - **El resultado visual:** Como Prometheus ya tenía configurado en su agenda raspar todas las IPs del laboratorio, al terminar el proceso detectará de inmediato a estos dos nuevos "trabajadores". Sin hacer absolutamente nada más, verás aparecer las gráficas de consumo de RAM y CPU de tus antiguas máquinas de Vagrant directamente en el videowall interactivo de Grafana de la UD04.

7. PRUEBAS MANUALES

Un ingeniero DevOps nunca asume que un despliegue ha sido exitoso simplemente porque no hubo errores en la pantalla; la infraestructura debe ser auditada. El laboratorio incorpora un sistema de validación forense para confirmar que todo el ecosistema es coherente y operativo.

```
bash scripts-laboratorio/08-verify-system.sh
```

- **¿Qué ocurre bajo el capó?**

```
alumno@alumno-virtualbox: /$ cd /opt/ud04-integracion-salt-monitoring/laboratorio$ bash scripts/laboratorio/08-verify-system.sh

UD04 - Verificación Integral del Sistema
(Chequeo de Salud y Conectividad: 40%)

[TEST] 1. Disponibilidad de Contenedores:
- Nodo vm-master: En ejecución (UP)
- Nodo vm-minion-1: En ejecución (UP)
- Nodo vm-minion-2: En ejecución (UP)
- Nodo cAdvisor: En ejecución (UP)
- Nodo blackbox-exporter: En ejecución (UP)

[TEST] 2. Direccionamiento de Red (Rango: 40%):
- IP Asignada -> vm-minion-2: 172.28.0.44/24
- IP Asignada -> vm-minion-1: 172.28.0.43/24
- IP Asignada -> vm-master: 172.28.0.40/24
- IP Asignada -> cAdvisor: 172.28.0.41/24
- IP Asignada -> blackbox-exporter: 172.28.0.42/24
```

- Este script hace las veces de un "monitor sintético externo". No asume nada; interroga a cada capa de la arquitectura para garantizar su viabilidad.
- **La explicación del arquitecto:** En lugar de comprobar simplemente si los procesos están iniciados, el script realiza pruebas funcionales de red y servicio:
 - **Estado de contenedores y red:** Consulta directamente la API de Docker para verificar que los cinco contenedores están en estado Running y correctamente anclados a la red devops_lab_net_ud04.
 - **Auditoría de SaltStack:** Ejecuta un test.ping masivo desde el contenedor maestro para confirmar que los Minions responden criptográficamente a través del bus ZeroMQ en tiempo real.
 - **Validación de Endpoints HTTP:** Utiliza curl para golpear las rutas críticas de salud (/-/healthy en Prometheus y /api/health en Grafana), asegurando que los demonios no solo están encendidos, sino listos para servir peticiones.
 - **Sondeos activos (Blackbox Exporter):** Dispara simulaciones de tráfico real. Ordena a Blackbox que lance un ping (ICMP) contra el Master y una petición HTTP contra cAdvisor, validando que el módulo de monitorización de red "hacia el exterior" está completamente operativo.

La prueba visual (Centro de mando)

La confirmación definitiva del éxito del laboratorio es la visualización de los datos. Abre tu navegador físico y visita los tres pilares del ecosistema:

1. **Las pantallas de control (Grafana):** 🖱️ <http://172.28.0.40:3000>
 - **Credenciales:** Usuario: admin / Contraseña: admin.
 - **Objetivo:** Ve al menú lateral "Dashboards" y abre el panel "**UD04 - System Overview**". Si ves gráficas de líneas de colores moviéndose, acabas de confirmar que la base de datos temporal, el Datasource y la inyección por API REST han funcionado a la perfección.
2. **El motor de búsqueda (Prometheus):** 🖱️ <http://172.28.0.40:9090>
 - **Objetivo:** Navega en el menú superior a Status > Targets. Aquí verás la "agenda" de Prometheus. Si todos los endpoints (**Node Exporters**, **cAdvisor**, **Blackbox**) aparecen en color verde (UP), significa que el modelo de extracción (Pull) está sorteando correctamente las redes de Docker y el puente de tu Host.
3. **El microscopio de Docker (cAdvisor):** 🖱️ <http://172.28.0.41:8080>
 - **Objetivo:** Esta es la interfaz pura de cAdvisor. Al navegar por ella, podrás ver métricas profundas a nivel de Kernel y Cgroups sobre cómo cada contenedor Docker individual está consumiendo los recursos de la máquina física.

8. OPERACIONES DE LIMPIEZA

Para devolver el sistema anfitrión a su estado original, no borres contenedores a mano. Usa las herramientas provistas para una limpieza profunda y ordenada:

1. Destrucción de la Infraestructura:

```
bash 98-destroy-infra.sh
```

- Fuerza la eliminación de los 5 contenedores, elimina la red interna de Docker (devops_lab_net_ud04), borra los volúmenes persistentes y purga los archivos de estado (.tfstate) de Terraform.

2. Demolición de la Red Puente:

```
bash 99-destroy-network.sh
```

- Baja y elimina la interfaz física **LabCEFIRE** y el nodo dum0 del Kernel de tu sistema.

9. PROBLEMAS COMUNES

En la ingeniería de sistemas reales, las infraestructuras complejas a veces fallan. La diferencia entre un simple operador y un arquitecto DevOps es la capacidad de aislar y diagnosticar el error a bajo nivel. Si te encuentras con un obstáculo durante el laboratorio, revisa estos tres sospechosos habituales:

● Colisión de puertos (Contenedores en bucle de reinicio):

- **El síntoma:** La fase de Terraform lanza un error de "Bind", o ejecutas docker ps y ves que contenedores como **cadvisor** o **vm-master** están iniciándose constantemente (Restarting).
- **El diagnóstico:** El culpable número uno es la capa de red de tu máquina física (Host). Al levantar los contenedores, Docker exige acceso exclusivo a los puertos. Si ya tienes un servidor Tomcat o una aplicación web local corriendo en tu PC físico por el puerto 8080, **cadvisor** colisionará frontalmente y el contenedor morirá. Lo mismo ocurrirá con el 3000 (**Grafana**) o el 9090 (**Prometheus**).
- **La solución:** Usa un escáner de red en tu sistema anfitrión (como **sudo netstat -tulpn | grep LISTEN** o **sudo lsof -i -P -n | grep LISTEN**) para cazar qué proceso oculto está secuestrando el puerto, mátalos y vuelve a lanzar el despliegue.

● Asimetría criptográfica (Minions "Zombies"):

- **El síntoma:** Ejecutas el comando de comprobación **salt '*' test.ping** y recibes un silencio absoluto, errores de timeout, o el mensaje Minion did not return.
- **El diagnóstico:** Esto significa que la confianza PKI se ha roto o desincronizado. El contenedor Minion está vivo y está enviando mensajes, pero el Master los descarta en la puerta de entrada porque la clave pública de ese Minion no figura

en su grupo de confianza (Accepted Keys).

- **La solución:** Entra interactivamente al cerebro de la infraestructura (***docker exec -it vm-master bash***). Ejecuta ***salt-key -L*** para auditar el estado de los certificados. Si ves las máquinas atrapadas en la sección Unaccepted Keys, fuerza la validación manual tecleando ***salt-key -A -y*** para restablecer el canal cifrado.

- **Ceguera de datos (Gráficas de Grafana vacías):**

- **El síntoma:** Consigues entrar al Dashboard "***UD04 - System Overview***", ves los marcos de los paneles perfectamente dibujados, pero no hay líneas de colores ni datos numéricos (aparece "No data").
- **El diagnóstico:** Este problema de observabilidad puede estar fallando en dos capas distintas. Por un lado, puede ser un fallo de recolección: Prometheus no puede leer el archivo ***prometheus.yml*** que le inyectó Terraform debido a conflictos de permisos, o los Exporters de las máquinas han muerto. Por otro lado, puede ser un fallo de enlace: la inyección del Datasource de Grafana a través de la API REST falló y Grafana no sabe dónde ir a buscar a Prometheus.
- **La solución:** No intentes adivinar. Aplica la metodología forense ejecutando el script ***bash scripts-laboratorio/08-verify-system.sh***. Este orquestador aislará matemáticamente en qué punto exacto se ha roto la cadena (te dirá si falla el endpoint de Prometheus, si el puerto de los Exporters está cerrado, o si Grafana no tiene conexión), permitiéndote atacar el problema de raíz.